

FPT ACADEMY INTERNATIONAL

CHƯƠNG TRÌNH KỸ THUẬT BÁN DẪN (SEMI 4)

BÁO CÁO ĐỒ ÁN

Kiểm Thử Thiết Kế Module Uart_Rx

Từ SystemVerilog OOP Đến UVM Framework

Sinh viên thực hiện: Đỗ Lập

Chuyên ngành: IC Design

Dự án: UART_RX Verification

FPT Academy International

Hà Nội – 2026

Mục Lục

Mục Lục	2
Tóm Tắt.....	3
1. Giới thiệu	4
1.1 Bối Cảnh.....	4
1.2 Thông Số Module.....	4
2. Mục Đích và Mục Tiêu	5
2.1 Mục Đích	5
2.2 Mục Tiêu Cụ Thể.....	5
3. Phương Pháp Thực Hiện.....	6
3.1 Kiến Trúc Testbench Tổng Quan	6
3.2 Phase 1 — Class-based Testbench (SystemVerilog OOP).....	6
3.3 Phase 2 — UVM Framework.....	6
3.4 Test Plan	7
4. Kết Quả Và Phân Tích.....	8
4.1 Phase 1 — Class-based Testbench.....	8
4.2 Phase 2 — UVM Framework.....	11
5. Kết Luận.....	14
5.1 Tổng Kết Kết Quả.....	14
5.2 Ý Nghĩa	14
5.3 Hướng Phát Triển	14

Tóm Tắt

Báo cáo này trình bày quá trình kiểm thử module UART RX được thiết kế bằng Verilog HDL, hoạt động với baud rate 115200 bps và clock 100MHz. Module sử dụng FSM 5 trạng thái và giao tiếp với CPU qua giao thức APB.

Quá trình kiểm thử được thực hiện theo 2 giai đoạn. Giai đoạn 1 sử dụng Class-based Testbench viết bằng SystemVerilog OOP với kiến trúc phân lớp gồm Transaction, Generator, Driver, Monitor và Scoreboard. Giai đoạn 2 nâng cấp lên UVM framework với uvm_driver, uvm_monitor, uvm_sequence và TLM port.

Kết quả kiểm thử: phát hiện 1 bug nghiêm trọng — FSM kẹt ở state STOP khi nhận STOP bit sai, đã báo cáo designer kèm đề xuất sửa. Functional coverage đạt 100% trên 5 bins được định nghĩa. Toàn bộ test case còn lại đều PASS ở cả 2 giai đoạn.

1. Giới Thiệu

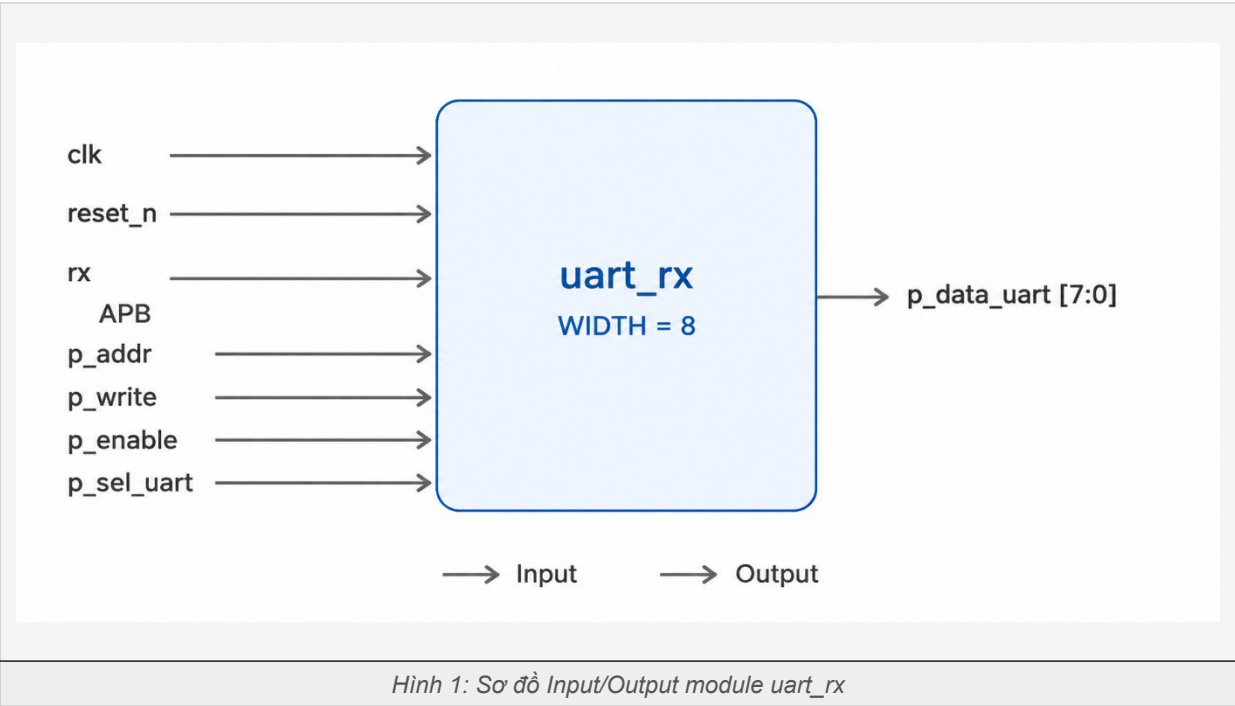
1.1 Bối Cảnh

Module UART RX là một trong những thành phần của hệ thống Mini SoC được thiết kế trong Project kì. Trong project đó, module đã được kiểm thử cơ bản bằng directed testbench đơn giản để xác nhận chức năng hoạt động.

Báo cáo này trình bày quá trình kiểm thử chuyên sâu hơn cho riêng module UART RX, áp dụng phương pháp Class-based Testbench và UVM framework — nhằm thực hành quy trình Design Verification có cấu trúc, coverage-driven theo chuẩn công nghiệp

1.2 Thông Số Module

Thông số	Giá trị
Clock	100 MHz
Baud Rate	115200 bps
B_COUNT	868 cycles/bit (100MHz / 115200)
Data width	8 bit, LSB first
Giao tiếp CPU	APB Bus
Thanh ghi	addr=0: RX_DONE_FLAG addr=4: RX_DATA
FSM	IDLE → START → RECIVE → STOP → DONE



2. Mục Đích và Mục Tiêu

2.1 Mục Đích

Mục đích của dự án là đảm bảo module UART RX hoạt động đúng theo spec thiết kế trước khi tích hợp vào hệ thống Mini SoC, đồng thời học và áp dụng 2 phương pháp kiểm thử từ cơ bản đến nâng cao.

2.2 Mục Tiêu Cụ Thể

Giai đoạn 1 — Class-based Testbench:

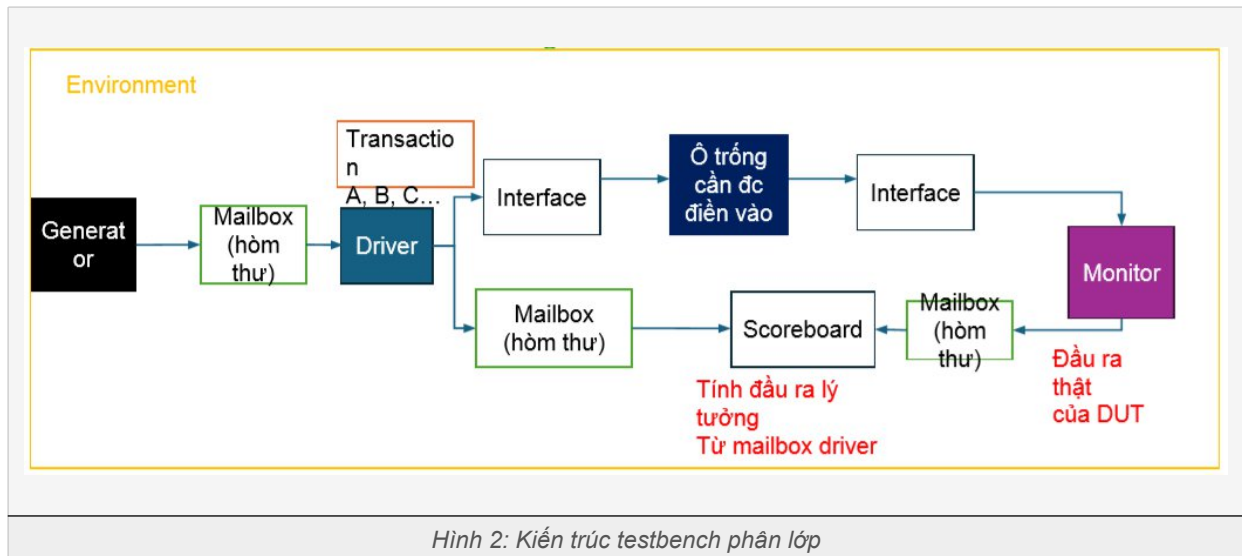
- Kiểm thử chức năng: module có nhận đúng data không, done_flag có hoạt động đúng không
- Kiểm thử giao thức APB: module chỉ xuất data khi tín hiệu APB hợp lệ
- Đạt 100% Functional Coverage trên các giá trị data quan trọng
- Phát hiện bug nếu có và báo cáo designer

Giai đoạn 2 — UVM Framework:

- Migrate testbench từ Class-based lên UVM framework chuẩn công nghiệp
- Re-verify toàn bộ test group bằng UVM methodology
- So sánh ưu nhược điểm giữa 2 phương pháp

3. Phương Pháp Thực Hiện

3.1 Kiến Trúc Testbench Tổng Quan



3.2 Phase 1 — Class-based Testbench (SystemVerilog OOP)

Testbench được xây dựng theo kiến trúc phân lớp với các thành phần độc lập, giao tiếp qua mailbox:

Thành phần	Nhiệm vụ	Ghi chú
Transaction	Định nghĩa data + constraint	rand data với dist weight cho các giá trị biên
Generator	Tạo data ngẫu nhiên	Gửi sang Driver và Scoreboard qua mailbox
Driver	Đẩy stimulus vào DUT	Chỉ gửi UART byte + điều khiển APB input
Monitor	Quan sát output DUT	Chỉ đọc p_data_uart, không chạm APB input
Scoreboard	So sánh + Coverage	So sánh data gốc (gen) vs data nhận (monitor)
Environment	Kết nối thành phần	Build + Run toàn bộ testbench

Luồng Mailbox:

```
Generator → [gen_drv_mailbox] → Driver → DUT
Generator → [gen_scb_mailbox] → Scoreboard (giữ data gốc)
DUT → Monitor → [mon_scb_mailbox] → Scoreboard (kết quả thực tế)
```

3.3 Phase 2 — UVM Framework

Testbench được migrate lên UVM framework với các thành phần chuẩn công nghiệp:

Class-based (Phase 1)	UVM (Phase 2)	Khác biệt chính
Transaction	uvm_sequence_item	Kế thừa uvm_object, có factory
Generator	uvm_sequence	Có p_sequencer, hỗ trợ virtual sequence
Driver	uvm_driver	Dùng TLM get_port thay mailbox
Monitor	uvm_monitor	Dùng TLM analysis_port
Scoreboard	uvm_scoreboard	Subscriber nhận từ analysis port
Mailbox	TLM port/export	Non-blocking, type-safe
Environment	uvm_env	UVM phasing: build→connect→run→report

3.4 Test Plan

Nhóm	Loại test	Mục tiêu
Functional	Correctness	Nhận đúng data theo spec
	Done Flag	done_flag lên 1 sau khi nhận xong
	Clear Flag	done_flag tự clear khi CPU đọc RX_DATA
	Reset	Reset đúng trạng thái, hoạt động lại sau reset
	Corner Case	Noise, false start, STOP bit = 0
Protocol	APB Handshake	p_sel/p_enable/p_write không hợp lệ → output = 0
Metrics	Functional Coverage	5 bins: 0x00, 0xFF, 0xAA, 0x55, others

4. Kết Quả Và Phân Tích

4.1 Phase 1 — Class-based Testbench

4.1.1 Functional Test — Correctness, Done Flag, Clear Flag

Flow test: kiểm tra done_flag=0 → gửi byte UART → chờ done_flag=1 → đọc RX_DATA → kiểm tra data → xác nhận flag về 0.

STT	Data gửi	Mô tả	Kỳ vọng	Kết quả
1	0x00	All zero	rx_data = 0x00, flag=1→0	PASS
2	0xFF	All one	rx_data = 0xFF, flag=1→0	PASS
3	0xAA	Xen kẽ 10101010	rx_data = 0xAA, flag=1→0	PASS
4	0x55	Xen kẽ 01010101	rx_data = 0x55, flag=1→0	PASS
5	Random	10 giá trị ngẫu nhiên	rx_data = data gửi	PASS

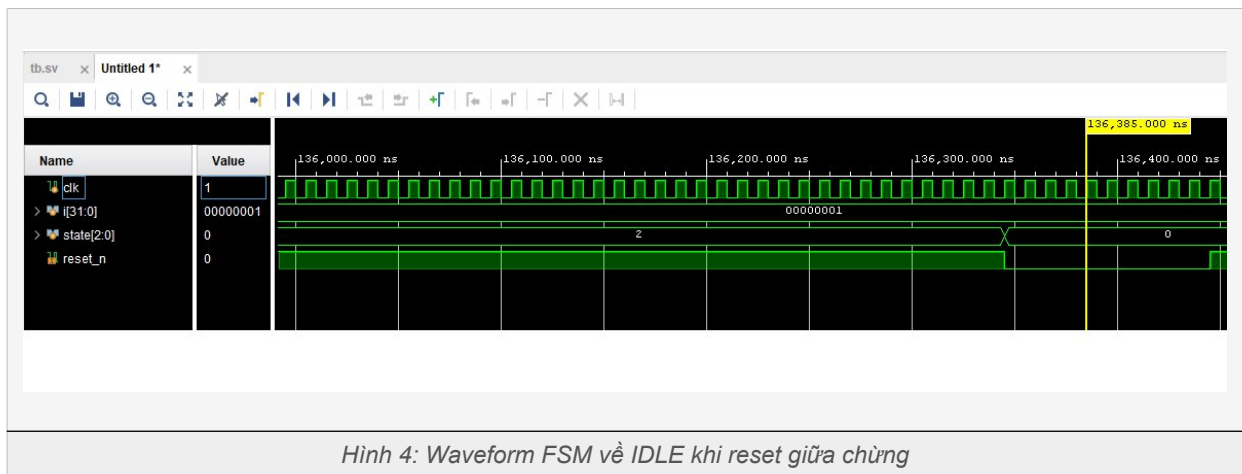
```
Test time: 3
=====
1. Kiểm tra output khi chưa có dữ liệu
[PASS] Expected: 0x00 | Got: 0x00
2. Kiểm tra out khi đã nhận dữ liệu - done flag
[PASS] Expected: 0x01 | Got: 0x01
3. Kiểm tra output so với byte uart
[PASS] Expected: 0xba | Got: 0xba
4. Kiểm tra done flag về 0
[PASS] Expected: 0x00 | Got: 0x00
=====
Test time: 4
=====
1. Kiểm tra output khi chưa có dữ liệu
[PASS] Expected: 0x00 | Got: 0x00
2. Kiểm tra out khi đã nhận dữ liệu - done flag
[PASS] Expected: 0x01 | Got: 0x01
3. Kiểm tra output so với byte uart
[PASS] Expected: 0x55 | Got: 0x55
4. Kiểm tra done flag về 0
[PASS] Expected: 0x00 | Got: 0x00
=====
```

Hình 3: Log terminal Correctness, Done Flag, Clear Flag test

4.1.2 Reset Test

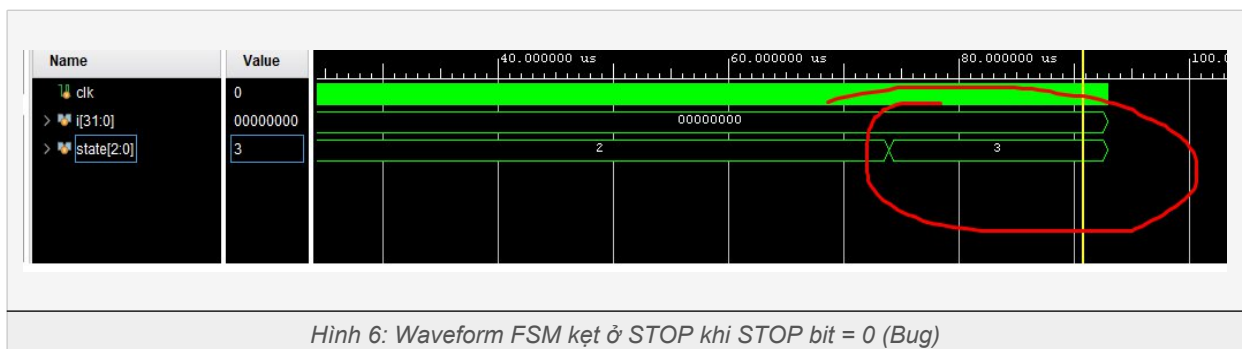
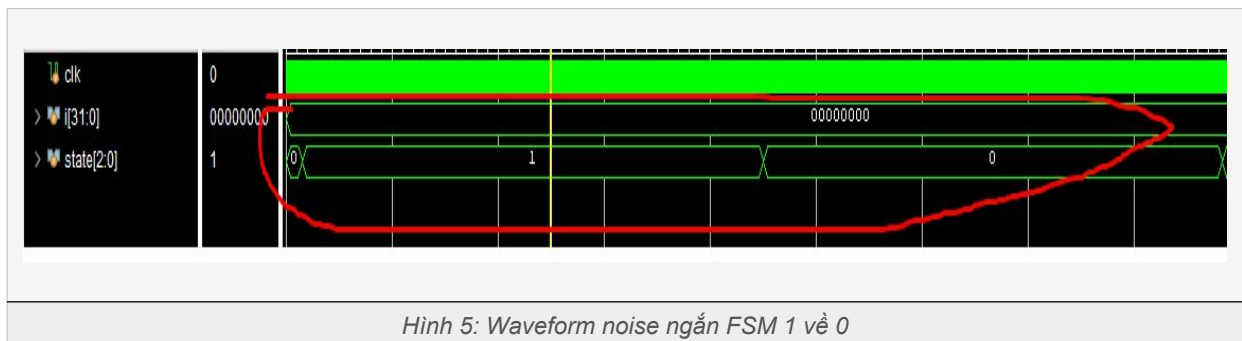
Gửi byte UART, kéo reset sau 3 bit, kiểm tra output về 0 và DUT hoạt động bình thường sau reset.

STT	Test case	Kỳ vọng	Kết quả
1	Reset giữa chừng sau 3 bit	done_flag=0, rx_data=0, FSM→IDLE	PASS



4.1.3 Corner Case Test

STT	Test case	Mô tả	Kỳ vọng	Kết quả
1	Noise ở IDLE	rx=0 chỉ 20 cycle rồi lên 1	FSM từ 1 về 0 : quay lại trạng thái IDLE	PASS
2	STOP bit = 0	Gửi frame lỗi STOP=0	FSM không bị kẹt	FAIL



4.1.4 Protocol Test — APB Handshake

STT	Điều kiện	Mô tả	Kỳ vọng	Kết quả
1	p_sel=0, p_enable=0	Không chọn module	p_data = 0x00	PASS
2	p_sel=1, p_enable=0	Thiếu enable	p_data = 0x00	PASS
3	p_sel=0, p_enable=1	Thiếu sel	p_data = 0x00	PASS
4	p_write=1	Write mode	p_data = 0x00	PASS
5	p_addr = invalid	Địa chỉ không hợp lệ	p_data = 0x00	PASS

4.1.5 Functional Coverage

Coverage chỉ áp dụng cho phần Correctness vì đây là phần dùng random — cần đo xem các giá trị quan trọng đã được random ra chưa. Các phần còn lại là directed test nên không cần coverage.

Bins	Giá trị	Số lần hit	Trạng thái
bins zero	0x00 (All zero)	≥ 1	HIT
bins max	0xFF (All one)	≥ 1	HIT
bins alt1	0xAA (10101010)	≥ 1	HIT
bins alt2	0x55 (01010101)	≥ 1	HIT
bins others	0x01~0xFE (ngẫu nhiên)	≥ 1	HIT
TỔNG			100%

```

=====
1. Kiểm tra output khi chưa có dữ liệu
[PASS] Expected: 0x00 | Got: 0x00
2. Kiểm tra out khi đã nhận dữ liệu - done flag
[PASS] Expected: 0x01 | Got: 0x01
3. Kiểm tra output số voi byte uart
[PASS] Expected: 0x3c | Got: 0x3c
4. Kiểm tra done flag về 0
[PASS] Expected: 0x00 | Got: 0x00
=====
Test time: 19
=====
1. Kiểm tra output khi chưa có dữ liệu
[PASS] Expected: 0x00 | Got: 0x00
2. Kiểm tra out khi đã nhận dữ liệu - done flag
[PASS] Expected: 0x01 | Got: 0x01
3. Kiểm tra output số voi byte uart
[PASS] Expected: 0x41 | Got: 0x41
4. Kiểm tra done flag về 0
[PASS] Expected: 0x00 | Got: 0x00
=====
COVERAGE: 100.00%
=====

```

Hình 7: Functional coverage report từ simulation tool

4.1.6 Bug Report

Bug ID	Vị trí	Mô tả	Mức độ	Trạng thái
BUG-01	State STOP	Khi STOP bit = 0, điều kiện rx==1 && count==B_COUNT-1 không bao giờ thỏa → FSM kẹt mãi ở STOP → module mất đồng bộ, nhận sai byte tiếp theo.	CRITICAL	Báo Designer

Đề xuất sửa Bug-01:

```
STOP: begin
    if(count_baud_tick == B_COUNT - 1) begin
        if(rx == 1) next_state = DONE; // STOP bit đúng
        else       next_state = IDLE;  // STOP bit sai → về IDLE
    end
end
```

4.2 Phase 2 — UVM Framework

Sau khi migrate sang UVM, toàn bộ test group được re-verify lại với kết quả tương đương Phase 1.

4.2.1 Kết Quả Re-verify

Nhóm test	Phase 1 (Class-based)	Phase 2 (UVM)
Functional Test	PASS (trừ BUG-01)	PASS (trừ BUG-01)
Protocol Test	5/5 PASS	5/5 PASS
Coverage	100%	100%

```
-----
0x00
.Project/1.Ki 2/1.Vivado/uart_rx/uart_rx.srcs/sim_1/new/tb.sv(295) @ 9466525000: uvm_test_top.env_o.sb [SCB] [PASS] CHECK1 Initial State
.Project/1.Ki 2/1.Vivado/uart_rx/uart_rx.srcs/sim_1/new/tb.sv(304) @ 9466525000: uvm_test_top.env_o.sb [SCB] [PASS] CHECK2 Done Flag
.Project/1.Ki 2/1.Vivado/uart_rx/uart_rx.srcs/sim_1/new/tb.sv(317) @ 9466525000: uvm_test_top.env_o.sb [SCB] [PASS] CHECK3 Data Valid
.Project/1.Ki 2/1.Vivado/uart_rx/uart_rx.srcs/sim_1/new/tb.sv(326) @ 9466525000: uvm_test_top.env_o.sb [SCB] [PASS] CHECK4 Flag Cleared
-----
.Project/1.Ki 2/1.Vivado/uart_rx/uart_rx.srcs/sim_1/new/tb.sv(68) @ 9466525000: uvm_test_top.env_o.agt.seqr@@bseq [base_seq] base_seq: Ins
.Project/1.Ki 2/1.Vivado/uart_rx/uart_rx.srcs/sim_1/new/tb.sv(156) @ 9466525000: uvm_test_top.env_o.agt.drv [driver] Sending byte: 0xaa
.Project/1.Ki 2/1.Vivado/uart_rx/uart_rx.srcs/sim_1/new/tb.sv(236) @ 9562145000: uvm_test_top.env_o.agt.mon [monitor] MON: flag_before=0x0
-----
0xaa
.Project/1.Ki 2/1.Vivado/uart_rx/uart_rx.srcs/sim_1/new/tb.sv(295) @ 9562145000: uvm_test_top.env_o.sb [SCB] [PASS] CHECK1 Initial State
.Project/1.Ki 2/1.Vivado/uart_rx/uart_rx.srcs/sim_1/new/tb.sv(304) @ 9562145000: uvm_test_top.env_o.sb [SCB] [PASS] CHECK2 Done Flag
.Project/1.Ki 2/1.Vivado/uart_rx/uart_rx.srcs/sim_1/new/tb.sv(317) @ 9562145000: uvm_test_top.env_o.sb [SCB] [PASS] CHECK3 Data Valid
.Project/1.Ki 2/1.Vivado/uart_rx/uart_rx.srcs/sim_1/new/tb.sv(326) @ 9562145000: uvm_test_top.env_o.sb [SCB] [PASS] CHECK4 Flag Cleared
-----
```

Hình 8: Log UVM

```

UVM_INFO E:/1.Chip design/2.Project/1.Ki 2/1.Vivado/uart_rx/uart_rx.srscs/sim_1/new/tb.sv(429) @ 9562145000: uvm_test_top [base_test] End of testcase
UVM_INFO E:/1.Vivado/Vivado/2024.2/data/system_verilog/uvm_1.2/xlnx_uvm_package.sv(19968) @ 9562145000: reporter [TEST_DONE] 'run' phase is ready to
=====
COVERAGE: 100.00%
=====
UVM_INFO E:/1.Vivado/Vivado/2024.2/data/system_verilog/uvm_1.2/xlnx_uvm_package.sv(13673) @ 9562145000: reporter [UVM/REPORT/SERVER]
--- UVM Report Summary ---

** Report counts by severity
UVM_INFO : 706
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0
** Report counts by id
[RNTST] 1
[SCB] 400
[TEST_DONE] 1
[UVM/COMP/NAMECHECK] 1
[UVM/RELNOTES] 1
[base_seq] 100
[base_test] 2
[driver] 100
[monitor] 100

```

Hình 9: Log terminal UVM

4.2.2 So Sánh 2 Phương Pháp

Tiêu chí	Class-based (Phase 1)	UVM (Phase 2)
Độ phức tạp	Đơn giản, dễ học	Phức tạp hơn, nhiều concept
Chuẩn công nghiệp	Không chuẩn UVM	Chuẩn IEEE 1800.2
Tái sử dụng	Khó tái sử dụng	Factory override, dễ mở rộng
Giao tiếp component	Mailbox	TLM port/export
Phù hợp	Học tập, project nhỏ	Project thực tế, công ty

5. Kết Luận

5.1 Tổng Kết Kết Quả

Nhóm test	Số test case	Kết quả
Functional Test	6 test cases	5 PASS / 1 FAIL (BUG-01)
Protocol Test	5 test cases	5 PASS / 0 FAIL
Coverage	5 bins	100% hit
TỔNG	11 test cases	10 PASS / 1 FAIL

5.2 Ý Nghĩa

- Đã kiểm thử đầy đủ module UART RX theo 3 nhóm: Functional, Protocol và Coverage
- Phát hiện 1 bug nghiêm trọng (FSM kẹt ở STOP) — đúng vai trò của DV engineer
- Áp dụng thành công 2 phương pháp từ cơ bản (Class-based) đến nâng cao (UVM)
- Functional coverage đạt 100% — xác nhận các giá trị quan trọng đã được test đầy đủ

5.3 Hướng Phát Triển

- Áp dụng UVM register model (uvm_reg) để kiểm thử APB register chính xác hơn
- Thêm coverage-driven verification: tự động chạy thêm test khi coverage thấp
- Mở rộng kiểm thử sang module UART TX và toàn bộ hệ thống Mini SoC
- Thêm formal verification để chứng minh toán học tính đúng đắn của FSM